# 2026-10-07 学习总结

# Map-Reduce、多模型路由与 RAG 质量调试

P1 RAG 项目 · Day 3 上午。从昨天 context-window 面试题的一道追问出发，亲手设计并跑通 map-reduce，中途撞上 429/503，手写多模型 fallback 路由，最后完成一条完整的 "debug RAG quality issue" 链。

本文档 = 中文总结 + English detailed version + 两道面试题（含中文思路与英文参考答案）。

## 第一部分：今日学习总结（中文）

### 1. Map-Reduce 设计

Map 把大任务拆成 10 个独立小任务（每 chunk 提炼要点，可并行）；Reduce 读完所有 map 输出再综合。关键设计来自 Haiyan 本人：每句陈述标注来源 chunk（evidence chain），这是防幻觉的最强手段之一——模型每写一句都得想清来源。

### 2. Chat vs generate_content 澄清

Haiyan 抓住了代码里的不一致。修正后原则：面向用户的问答默认 Chat（多轮、留历史）；pipeline 内部一次性调用用 generate_content（map 的 10 个独立任务硬套 Chat 需建 10 个 session，无收益）。选接口看场景，不看约定。

### 3. 429 配额墙

gemini-3.8-flash 免费 tier 限 5 req/min。加了退避重试，但并行重试在紧配额下 ping-pong 撞墙（A 用完配额 B 醒来又撞）→ 退回串行 + 节流（13s 间隔）。配额有三层：按分钟、按天、按模型。

### 4. 503 过载

和 429 是两种失败：429 是“你用太多”，503 是“服务器忙不过来”。503 用指数退避（5s→10s→20s→40s），工业标准做法。

### 5. 多模型 fallback 路由

手写版：503 直接换模型，429 等 60s 后换，404/下线直接跳过。三种路由策略：fallback（降级）、cost-based（便宜模型先答）、load balancing（分摊 QPS）。工业版是 LiteLLM Router（P5）。

### 6. 模型可用性实测

2.5-flash 新用户不可用、3.8-flash 当天高负载、只有 3.5-flash 可用。可用模型集合本身不稳定——这正是 fallback 存在的理由，而不是硬编码单个模型。

### 7. v1 答案评估：机制 A，内容 C+

citation 全、没编造、综合了 6 个 chunk；但太浅——“Use three metrics”，哪三个？丢了。

### 8. 改 prompt

map 要求保留细节（names / numbers / definitions，不许只写话题标签）；reduce 要求归类合并，不止罗列。

### 9. v2 答案评估：A-

有深度、有结构、诚实标注缺失（Aspects Not Covered）。缺失归因到 chunking：按空行切分把 “Three metrics:” 标题和它的列表切到了两个 chunk。工业界用 overlap / semantic chunking 就是为了解这个。

### 10. 亲手摸到的 eval 维度

faithfulness（没编造）vs completeness（答得全不全）；拒答率也是指标。over-refusal vs hallucination 是跷跷板，调平衡是 P3 的工作。

## Part 2: English Detailed Version

### 1. Map-reduce design

Split synthesis into Map (one independent extraction task per chunk, parallelizable) and Reduce (read all map outputs, then synthesize). The key design decision — citing the source chunk for every statement (evidence chain) — was the user's own proposal, and it is one of the strongest anti-hallucination mechanisms: the model must ground each sentence.

### 2. Chat vs generate_content

The user caught an inconsistency in my code. Corrected principle: user-facing Q&A defaults to Chat (multi-turn, keeps history); one-shot calls inside a pipeline use generate_content (forcing 10 independent map tasks into Chat would require 10 separate sessions with no benefit). Choose the interface by whether the call needs history, not by convention.

### 3. The 429 quota wall

gemini-3.8-flash free tier allows 5 req/min. Backoff-and-retry was added, but parallel retries under a tight quota livelock (workers ping-pong: A consumes the refreshed quota just as B wakes up). Fell back to serial execution with 13s pacing. Quotas have three dimensions: per-minute, per-day, per-model.

### 4. 503 overload

A different failure mode from 429 — 429 means 'you are calling too much', 503 means 'the server is overloaded'. 503 gets exponential backoff (5s → 10s → 20s → 40s), the industry standard.

### 5. Multi-model fallback routing

Hand-rolled version: switch models immediately on 503, wait-then-switch on 429, skip on 404/model retired. Three routing strategies exist: fallback (degradation order), cost-based (cheap model first, escalate on low confidence), and load balancing (spread QPS). The production tool is LiteLLM Router (covered in P5).

### 6. Empirical model availability

gemini-2.5-flash is unavailable to new users; gemini-3.8-flash was overloaded all day; only gemini-3.5-flash worked. The available-model set is itself unstable — which is exactly why fallback routing exists instead of hardcoding one model.

### 7. v1 answer review: mechanics A, content C+

All statements cited, no fabrication, synthesized across 6 chunks; but too shallow — 'Use three metrics'. Which three? The details were lost in map-phase extraction.

### 8. Prompt fixes

Map must preserve details (names, numbers, definitions — never reduce to topic labels); reduce must organize into a coherent structure and merge overlapping points, not just list.

### 9. v2 answer review: A-

Deep, structured, and honestly flags gaps ('Aspects Not Covered'). The remaining gaps were attributed to chunking, not map-reduce: blank-line splitting had separated the 'Three metrics:' header from its list into

different chunks. This is why production uses overlap or semantic chunking.

## 10. Eval dimensions touched firsthand

faithfulness (no fabrication) vs completeness/answer depth; refusal rate is itself a metric. Over-refusal vs hallucination is a seesaw — tuning it is P3's job.

## 第三部分：面试题 + 参考答案

### 昨日题（context-window）

题目：你的 RAG 系统检索到了 20 个相关 chunk，每个约 500 token，模型的 context window 是 8k。你会把 20 个全塞进 prompt 吗？

**中文思路：** 先算账（20×500=10k > 8k，物理塞不下），再讲“塞得下也不该塞”（lost in the middle / noise 稀释 / 成本延迟），方案是“宽进窄出”（retrieve 宽 → rerank → top-k，k 靠 eval 定）。追问1用 eval 曲线证明 k；追问2讲大 window 的正确用法是给长输入留余量；追问3就是今天的 map-reduce。

**English answer:** "No — the math alone rules it out: 20 × 500 ≈ 10,000 tokens exceeds an 8k window before the system prompt and answer. But even with headroom I wouldn't stuff all 20 in: the 'lost in the middle' effect (Liu et al., 2023), noise diluting signal, and cost/latency scaling. My approach is retrieve-wide, generate-narrow: retrieve top-20/50, rerank, feed only top-k, with k tuned as a hyperparameter on a golden eval set. On proving k suffices: measure retrieval recall and answer quality vs k, pick the knee of the curve. On bigger windows: they buy headroom for genuinely long inputs, not license to dilute signal. On cross-chunk synthesis: map-reduce — extract per-chunk points independently (parallel), then synthesize with per-statement citations."

### 今日题（RAG system design）

题目：Design a RAG system for a customer support chatbot. How do you evaluate it?

**中文思路：** 先画全链路——ingest（chunking → embedding → vector DB + BM25 hybrid）、online（query rewrite → hybrid retrieve top-50 → rerank → top-5 → guardrail prompt → LLM → citation）、eval 回环。问题后半句是重点：offline（golden set + RAGAS：faithfulness / answer relevancy / context precision-recall + LLM-as-judge）+ online（用户点赞/点踩、拒答率、延迟）。追问1先定位瓶颈再动旋钮（rerank 换小、减候选、缓存、流式）；追问2分层 debug（检索错看 sources / 生成幻觉看 faithfulness / prompt 弱加约束）；追问3先算账（QPS、索引规模、token 成本）再给架构（HNSW、缓存、batch、多模型路由）。

**English answer:** "I'd design it in three layers. Ingest: chunk documents with overlap, embed, and store in a vector DB alongside a BM25 index for hybrid retrieval. Online: rewrite the query, hybrid-retrieve top-50, rerank with a cross-encoder down to top-5, then generate with a guardrailed prompt — context-only, per-statement citations, explicit refusal when evidence is missing. Evaluation is half the system: offline, a golden Q&A set scored with RAGAS (faithfulness, answer relevancy, context precision/recall) plus LLM-as-judge, run as regression on every change; online, thumbs up/down, refusal rate, and latency percentiles. On 500ms latency creep: locate the bottleneck first — profile retrieve vs rerank vs LLM. Knobs: smaller reranker or fewer candidates, cache frequent questions, stream the response, parallelize independent calls. On confidently-wrong answers: debug by layer — check the cited sources (retrieval wrong?), check faithfulness (generation hallucinating despite good sources?), check prompt constraints. Fixes: enforce citations, add a

confidence threshold with fallback to human, and add the failing case to the golden set. On 10K queries/day over 1M docs: do the math first — ~0.12 QPS average with peaks, 1M docs means precomputed embeddings in an HNSW index. Architecture: hybrid retrieval, aggressive caching of head queries, batch embedding for ingest, and multi-model routing for cost/latency control."